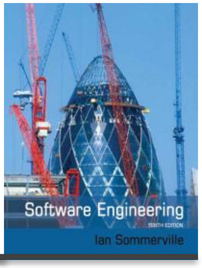
Chapter 5 – System Modeling

Topics covered



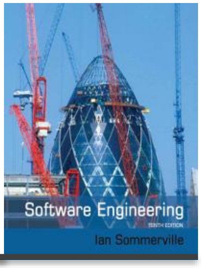
✧ Context models

✧ Interaction models

✧ Structural models

✧ Behavioral models

✧ Model-driven engineering



System modeling

- System modeling is the process of developing abstract models of a system, with each model presenting a different view or perspective of that system.
- System modeling has now come to mean representing a system using some kind of graphical notation, which is now almost always based on notations in the Unified Modeling Language (UML).
- System modelling helps the analyst to understand the functionality of the system and models are used to communicate with customers.

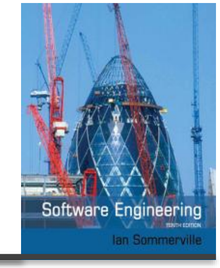
Existing and planned system models

- ✧ Models of the existing system are used during requirements engineering. They help clarify what the existing system does and can be used as a basis for discussing its strengths and weaknesses.
  - Lead to requirements for the new system.
- ✧ Models of the new system are used during requirements engineering to help explain the proposed requirements to other system stakeholders. Engineers use these models to discuss design proposals and to document the system for implementation.
- ✧ In a model-driven engineering process, it is possible to generate a complete or partial system implementation from the system model.

System perspectives

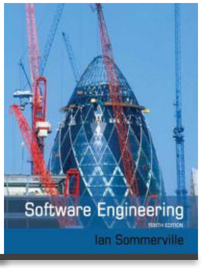
- An **external perspective**, where you model the **context or environment** of the system.
- An **interaction perspective**, where you **model the interactions between a system** and its environment, or between the components of a system.
- A **structural perspective**, where you **model the organization of a system or the structure** of the data that is processed by the system.
- A **behavioral perspective**, where you **model the dynamic behavior of the system** and how it responds to events.

UML diagram types



- **Activity diagrams**, which show the activities involved in a process or in data processing .
- Use case diagrams, which show the interactions between a system and its environment.
- **Sequence diagrams**, which show interactions between actors and the system and between system components.
- **Class diagrams**, which show the object classes in the system and the associations between these classes.
- **State diagrams**, which show how the system reacts to internal and external events.

Use of graphical models



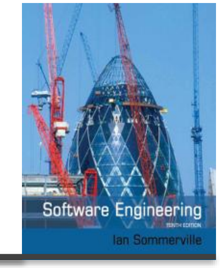
- As a **means of facilitating discussion** about an existing or proposed system
  - Incomplete and incorrect models are OK as their role is to support discussion.
- As a **way of documenting an existing system**
  - Models should be an accurate representation of the system, but need not be complete.
- As a **detailed system description** that can be used to generate a system implementation
  - Models have to be both correct and complete.

Context models

Context models

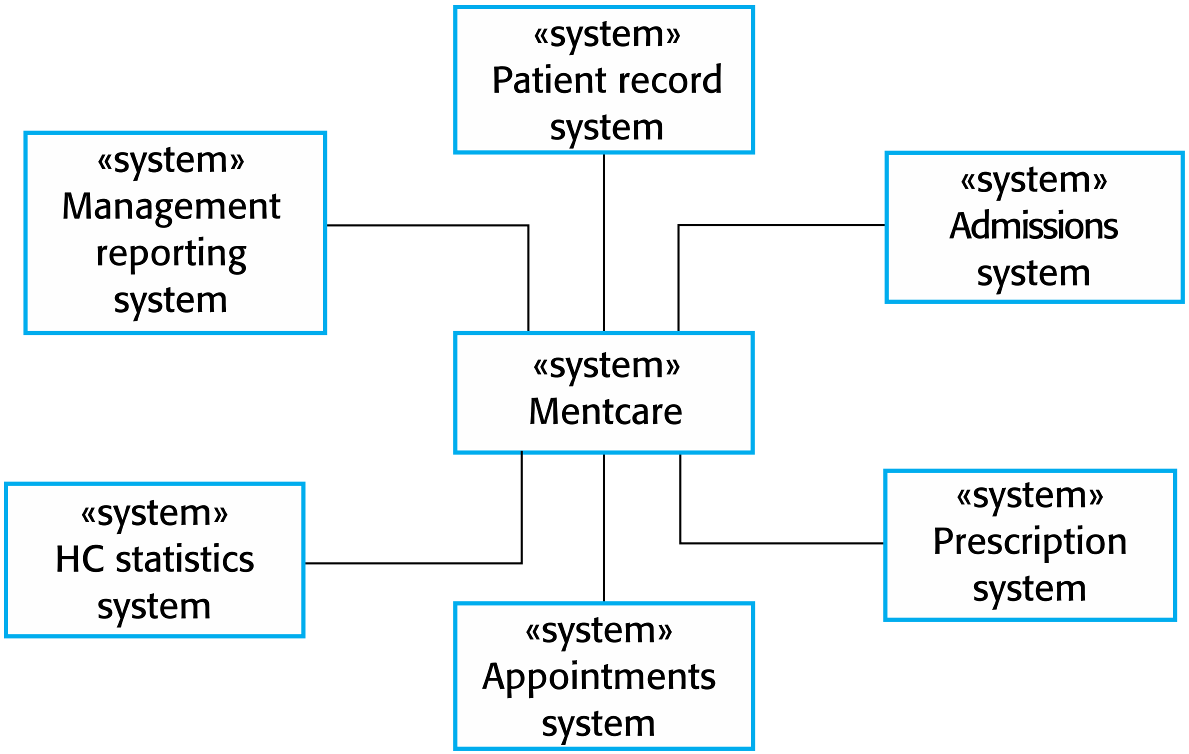
- **Context models** are used to illustrate the operational context of a system - they show what lies outside the system boundaries.
- Social and organizational concerns may affect the decision on where to position system boundaries.
- **Architectural models** show the system and its relationship with other systems.

System boundaries



- **System boundaries** are established to define what is **inside** and what is **outside** the system.
  - They show other systems that are used or depend on the system being developed.
- The **position of the system boundary** has a profound **effect on the system requirements**.

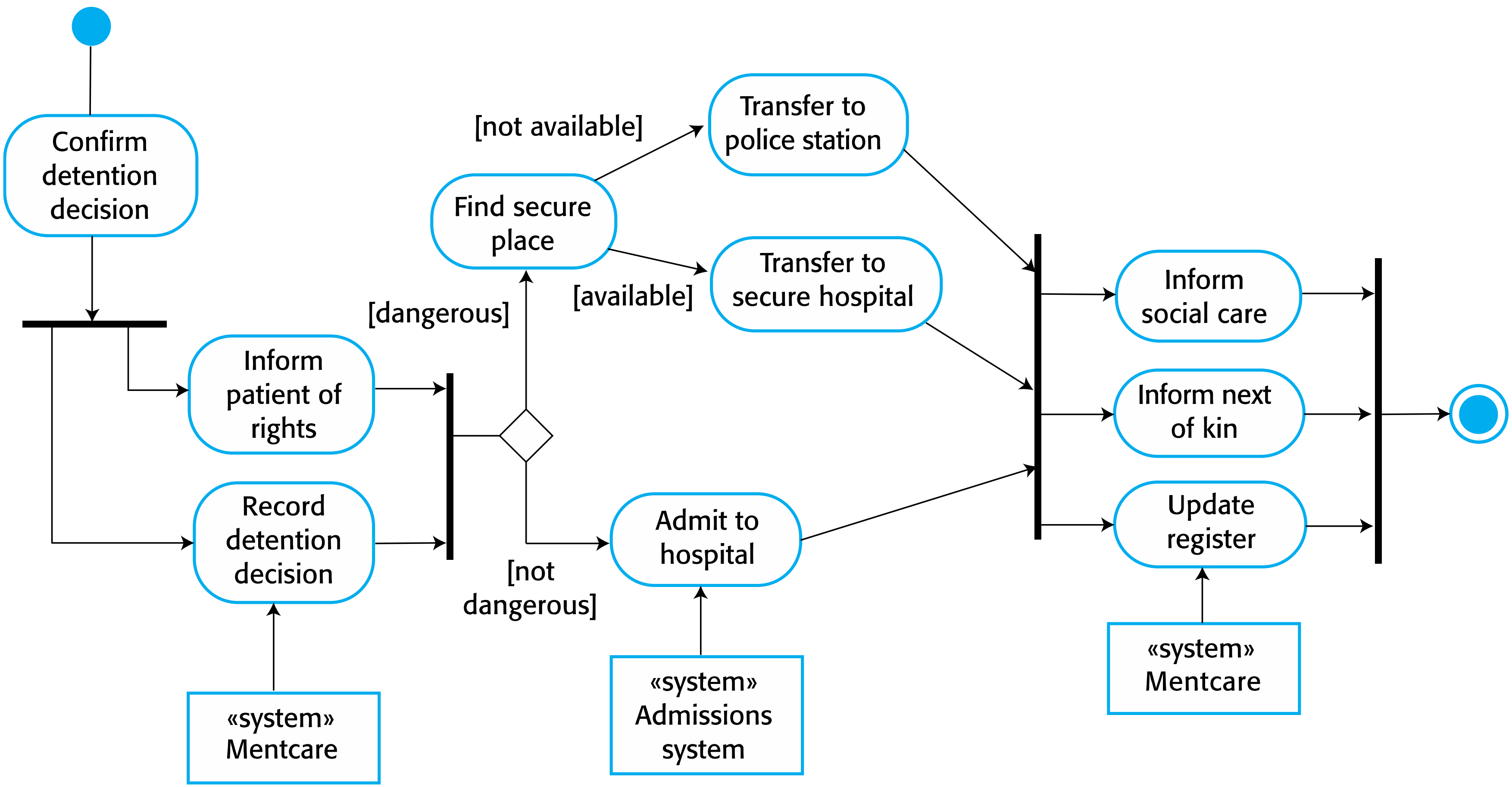
The context of the Mentcare system

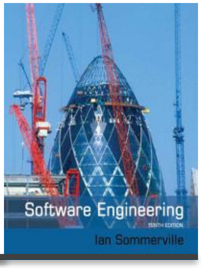


Process perspective

- **Context models** simply show the **other systems in the environment**, not how the system being developed is used in that environment.
- **Process models** reveal how the system being developed is **used in broader business processes**.
- **UML activity diagrams** may be used to **define business processes and models.**

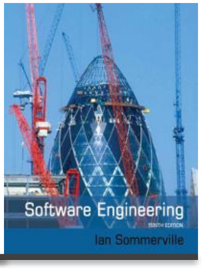
Process model of involuntary detention





Interaction models

Interaction models



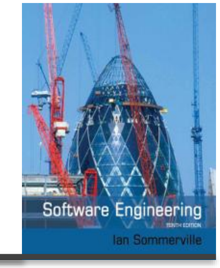
✧ Modeling **user interaction** is important as it helps to identify user requirements.

✧ Modeling **system-to-system interaction** highlights the communication problems that may arise.

✧ Modeling **component interaction** helps us understand if a proposed system structure is likely to **deliver the required system performance and dependability**.

✧ Use case diagrams and sequence diagrams may be used for interaction modeling.

Use case modeling



✧ Use cases were developed

- originally to support requirements elicitation and
- now incorporated into the UML.

✧ Each use case represents a **discrete task** that involves external interaction with a system.

✧ Actors in a use case may be

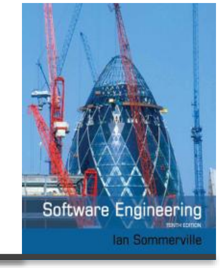
- people
- other systems.

✧ Represented diagrammatically to provide an overview of the use case and in a more detailed textual form.

Transfer-data use case

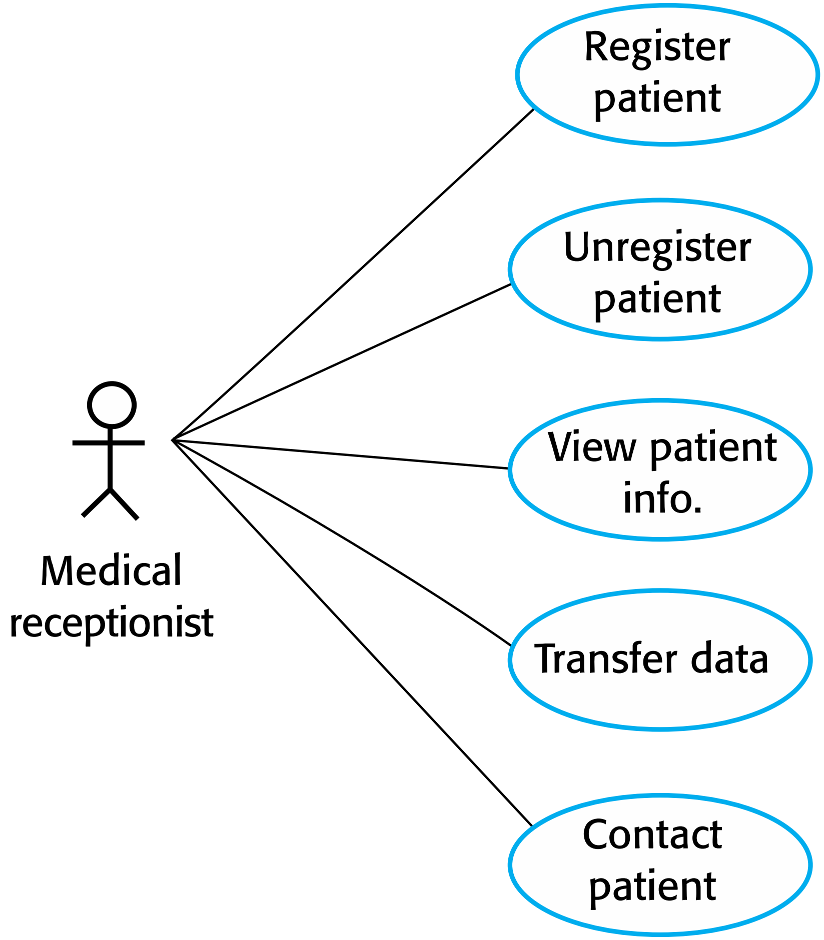
✧ A use case in the Mentcare system

Tabular description of the 'Transfer data' use-case

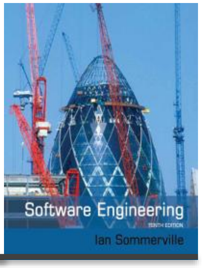


MHC-PMS: Transfer data	
Actors	Medical receptionist, patient records system (PRS)
Description	A receptionist may transfer data from the Mentcase system to a general patient record database that is maintained by a health authority. The information transferred may either be updated personal information (address, phone number, etc.) or a summary of the patient's diagnosis and treatment.
Data	Patient's personal information, treatment summary
Stimulus	User command issued by medical receptionist
Response	Confirmation that PRS has been updated
Comments	The receptionist must have appropriate security permissions to access the patient information and the PRS.

Use cases in the Mentcare system involving the role 'Medical Receptionist'



Sequence diagrams



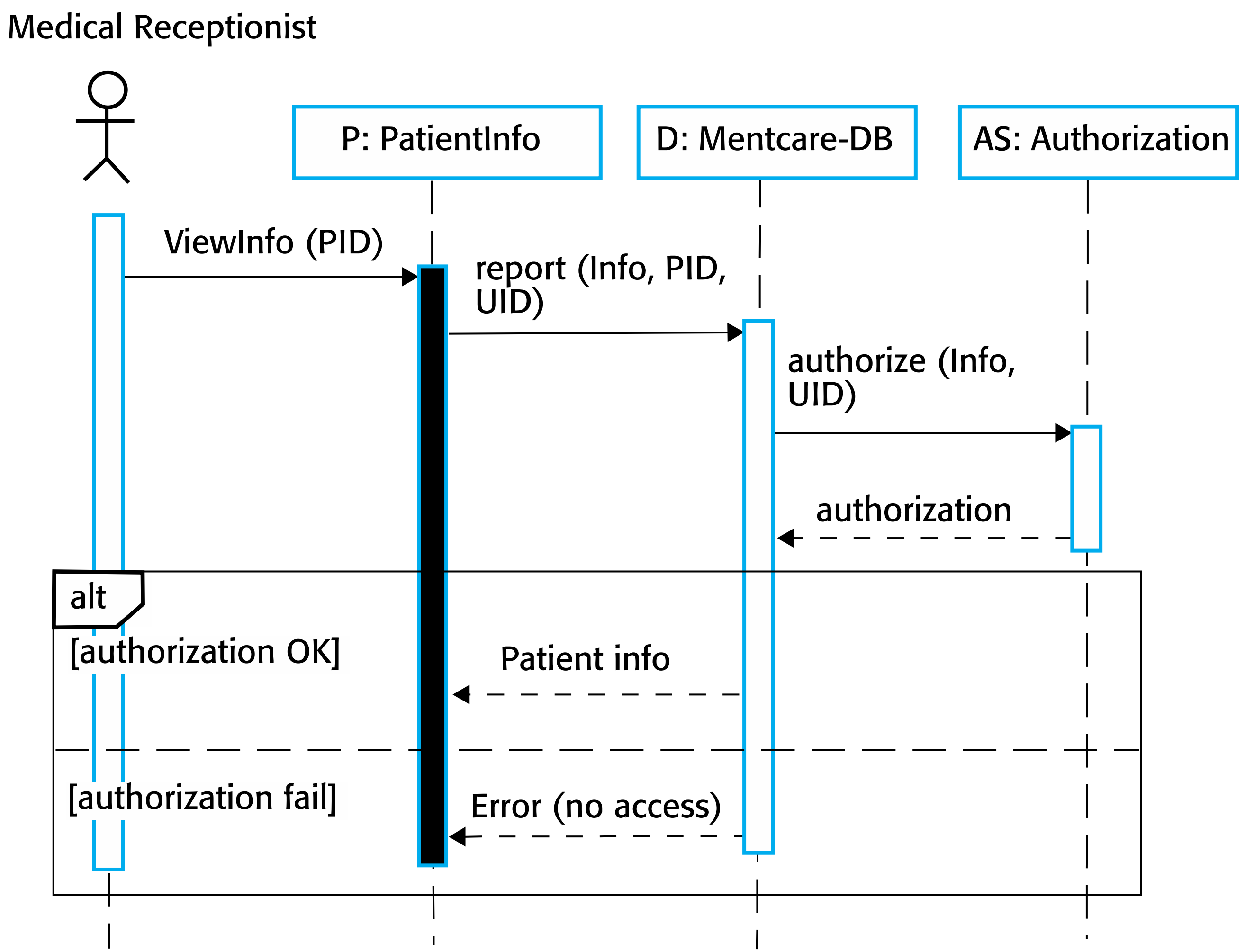
✧ **Sequence diagrams**:

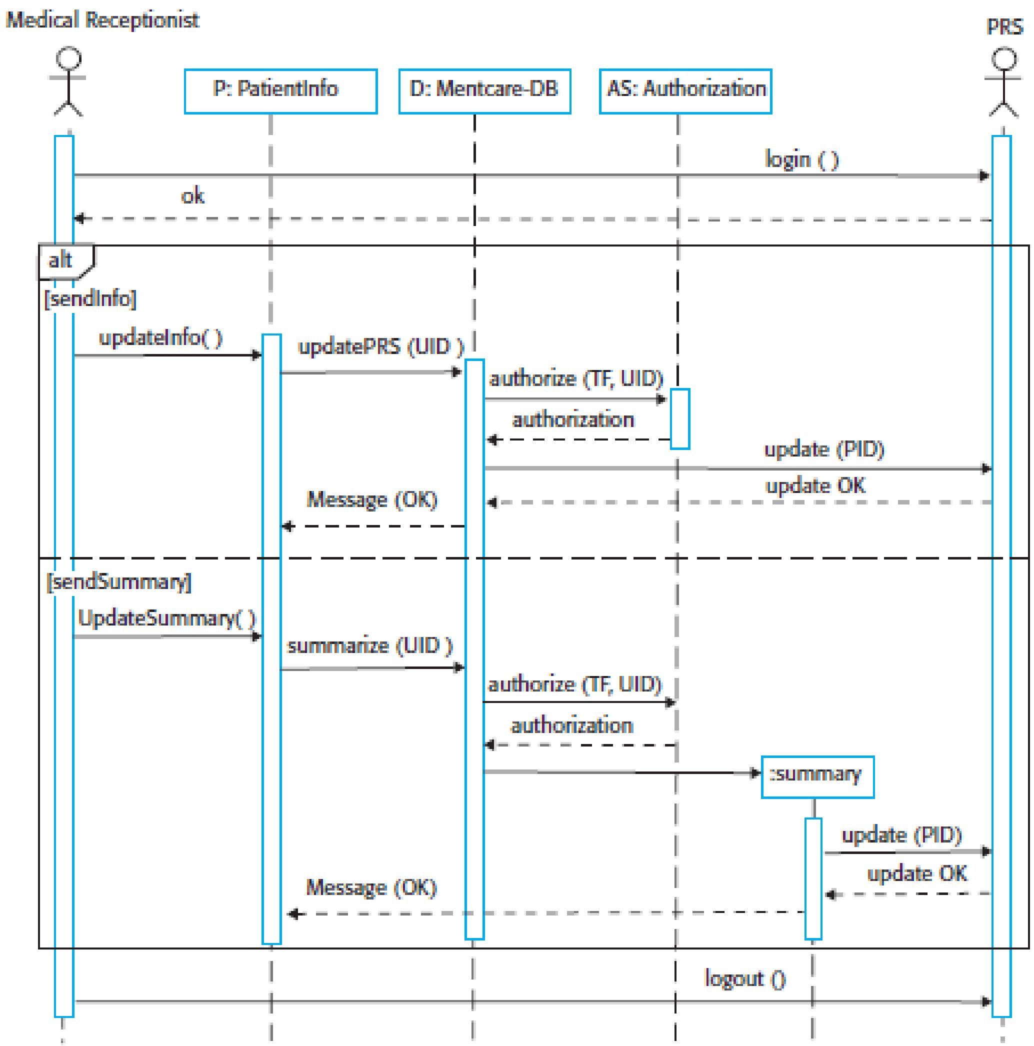
- part of the UML
- model the interactions between the actors and the objects within a system.
- shows the sequence of interactions that take place during a particular use case or use case instance.

✧ The **objects** and **actors** involved are listed along the top of the diagram, with a dotted line drawn vertically from these.

✧ Interactions between objects are indicated by annotated arrows.

Sequence diagram for View patient information

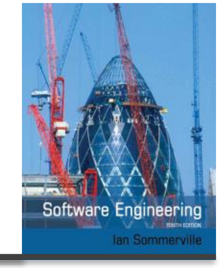




Sequence diagram for Transfer Data

Structural models

Structural models



✧ Structural models of software

- display the organization of a system in terms of the components that make up that system and their relationships.

✧ Structural models may:

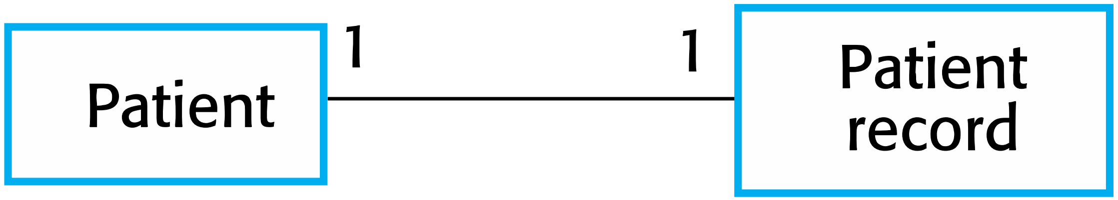
- **Static models** - show the structure of the system design
- **Dynamic models** - show the organization of the system when it is executing.

✧ You create structural models of a system when you are discussing and designing the system architecture.

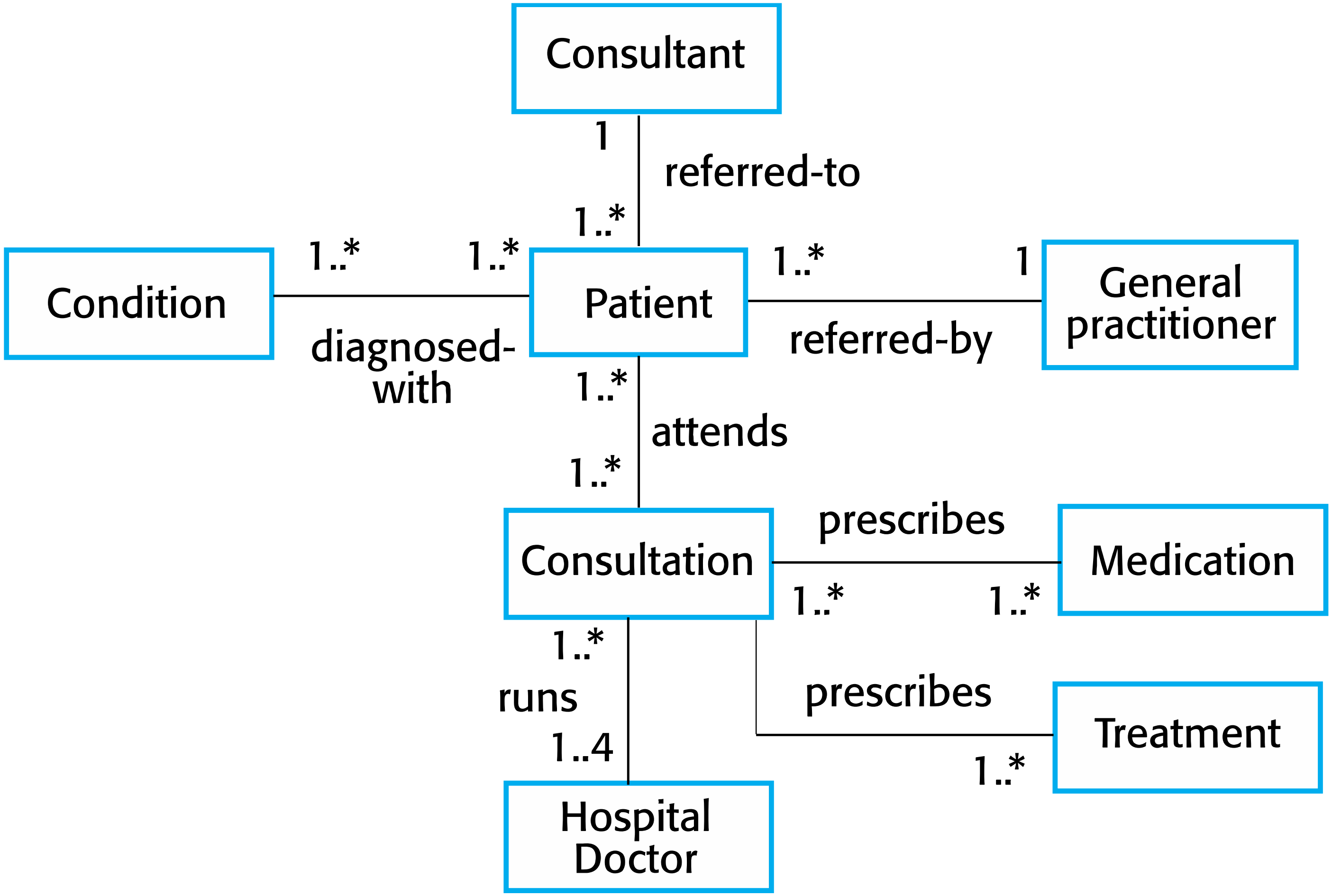
Class diagrams

- **Class diagrams** are
  - used **when developing an object-oriented system model** to show the classes in a system and the associations between these classes.
- An **object class** - general definition of one kind of system object.
- An **association** - a link between classes that indicates that there is some relationship between these classes.
- **Objects** - represent something in the real world
  - e.g., a patient, a prescription, a doctor, etc.

UML classes and association



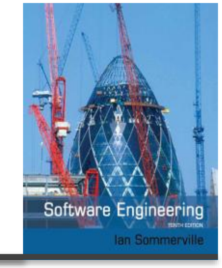
Classes and associations in the MHC-PMS



The Consultation class

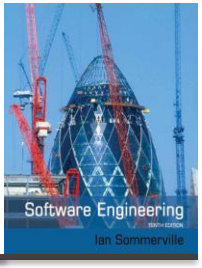
Consultation
Doctors Date Time Clinic Reason Medication prescribed Treatment prescribed Voice notes Transcript ...
New ( ) Prescribe ( ) RecordNotes ( ) Transcribe ( ) ...

Generalization



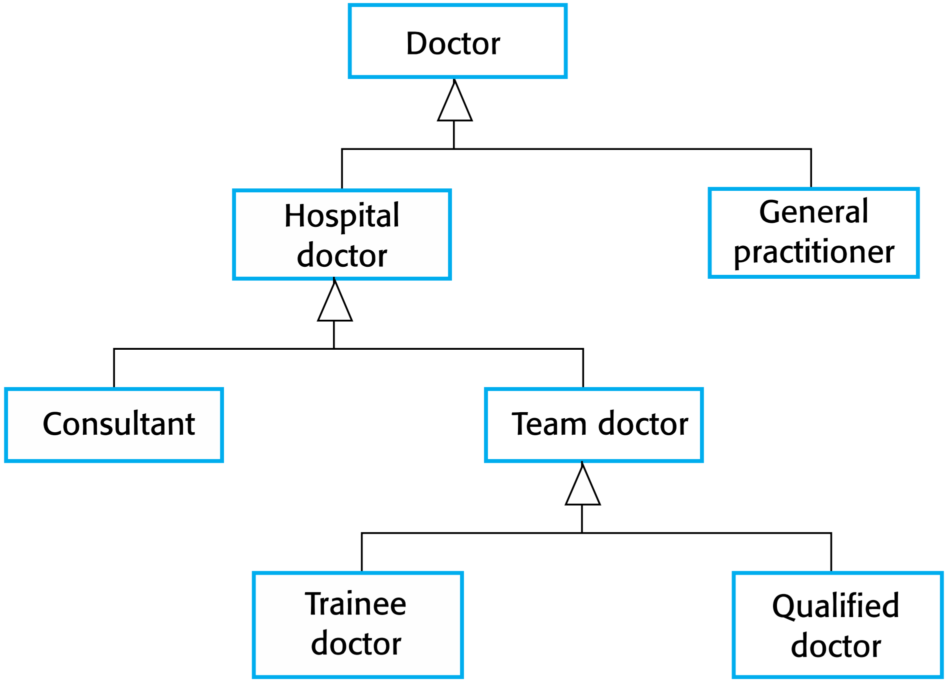
- **Generalization** – a technique used to manage complexity.
- Rather than learn the detailed characteristics of every entity that we experience, we place these entities in more general classes (animals, cars, houses, etc.) and learn the characteristics of these classes.
- This allows us to infer that different members of these classes have some common characteristics.

Generalization

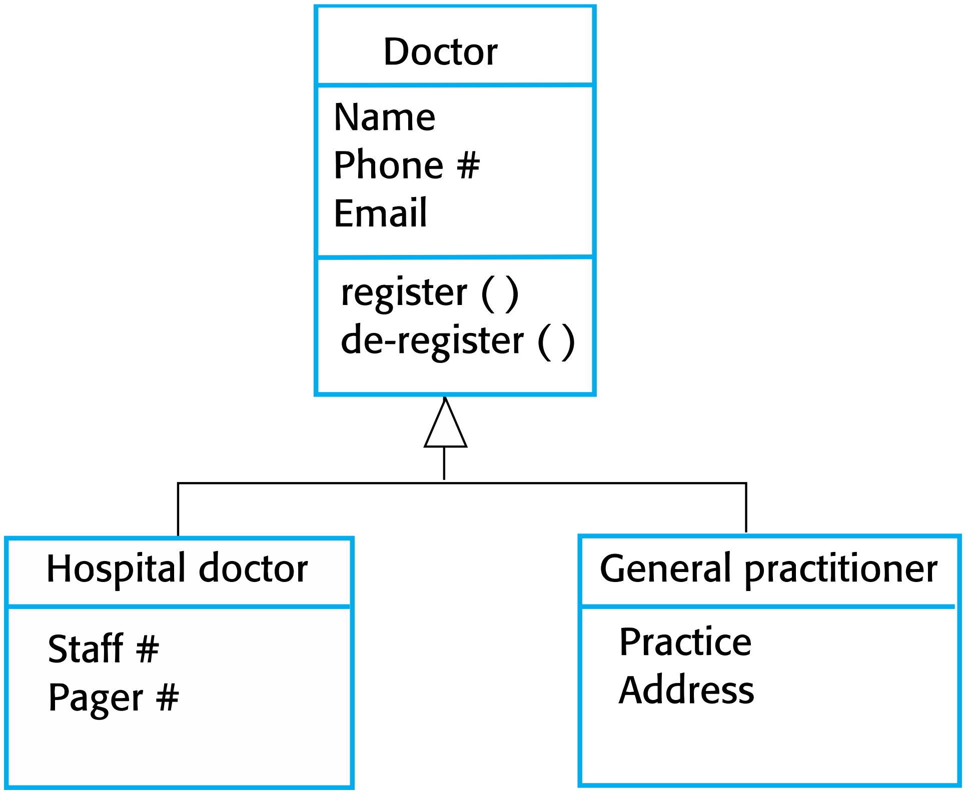


- In **modeling systems**, it is often useful to examine the classes in a system to see if there is scope for generalization.
- In **object-oriented languages**, such as **Java**, generalization is implemented using the <u>class inheritance mechanisms</u> built into the language.
- In a generalization, the attributes and operations associated with higher-level classes are also associated with the lower-level classes.
- The lower-level classes are subclasses that inherit the attributes and operations from their superclasses..

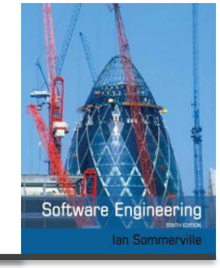
A generalization hierarchy



A generalization hierarchy with added detail

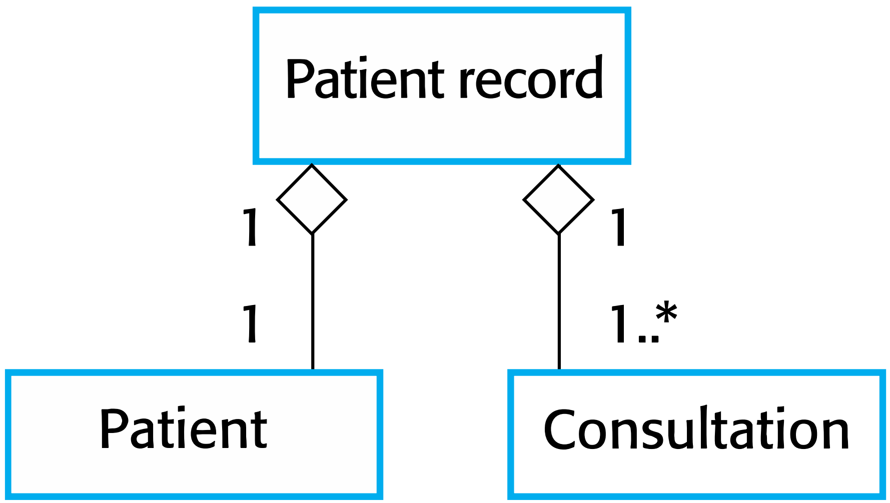


Object class aggregation models



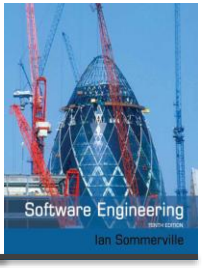
- ✧ An aggregation model
  - classes that are collections are composed of other classes.
- ✧ Aggregation models are similar to the part-of relationship in semantic data models.

The aggregation association



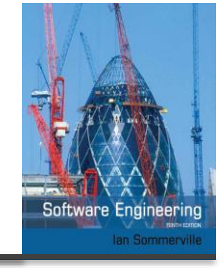
Behavioral models

Behavioral models



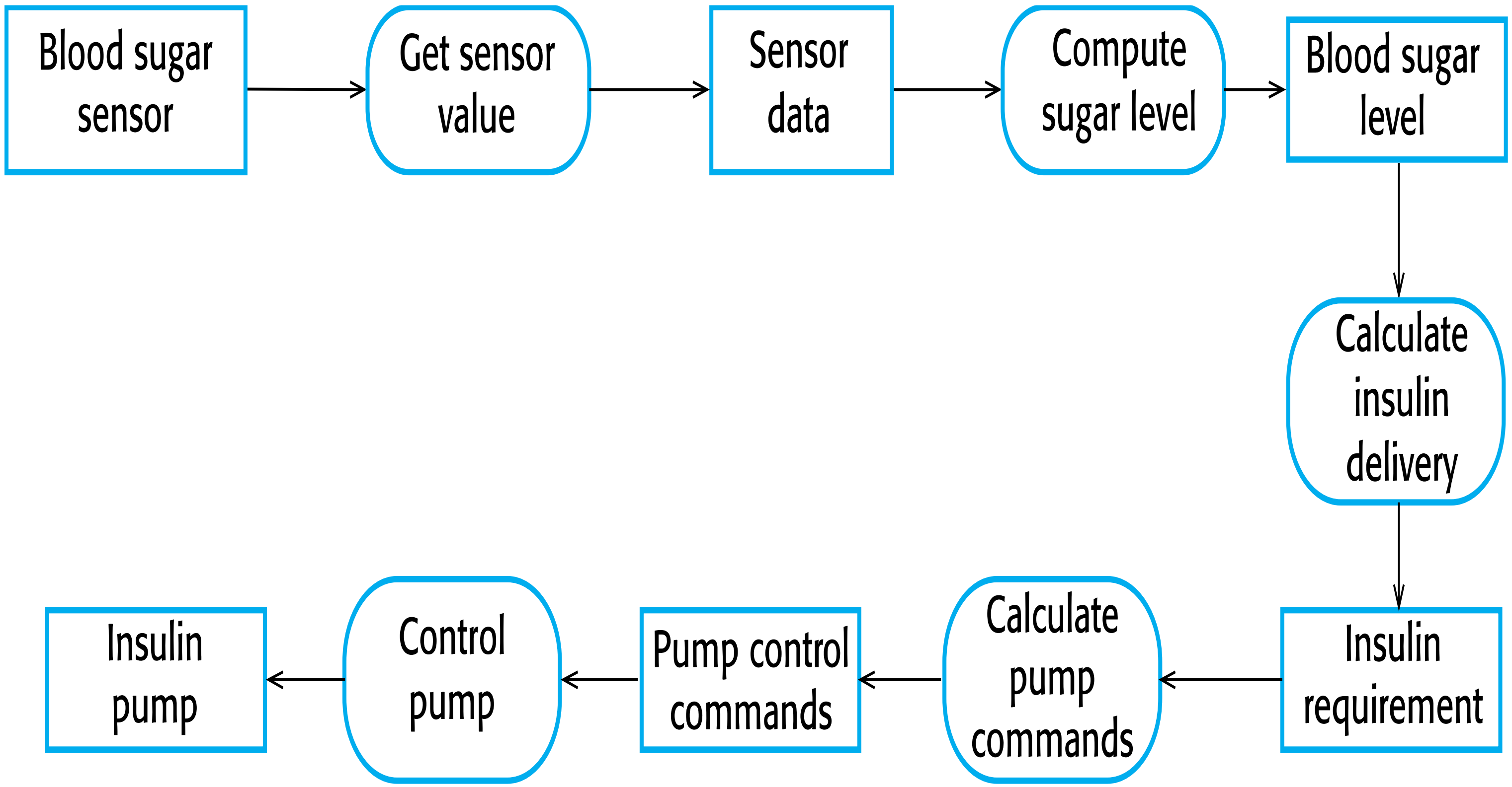
- ✧ **Behavioral models** are models of the dynamic behavior of a system as it is executing.
- ✧ They show what happens or what is supposed to happen when a system responds to a stimulus from its environment.
- ✧ of two types of stimuli:
  - Data: Some data arrives that has to be processed by the system.
  - Events: Some event happens that triggers system processing.

Data-driven modeling



- ✧ Many business systems are data-processing systems that are **primarily driven by data**.
  - controlled by the data input to the system, with relatively little external event processing.
- ✧ **Data-driven models** – a sequence of actions involved in processing input data and generating an associated output.
  - useful during the analysis of requirements as they can be used to show end-to-end processing in a system.

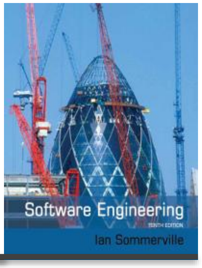
An activity model of the insulin pump's operation



Event-driven modeling

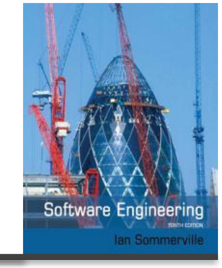
- ✧ **Real-time systems** are often event-driven, with minimal data processing.
  - For example, a landline phone switching system responds to events such as '**receiver off hook**' by **generating a dial tone**.
  - how a system responds to external and internal events.
- ✧ It is based on the assumption:
  - A system has a finite number of states, and events (stimuli) may cause a transition from one state to another.

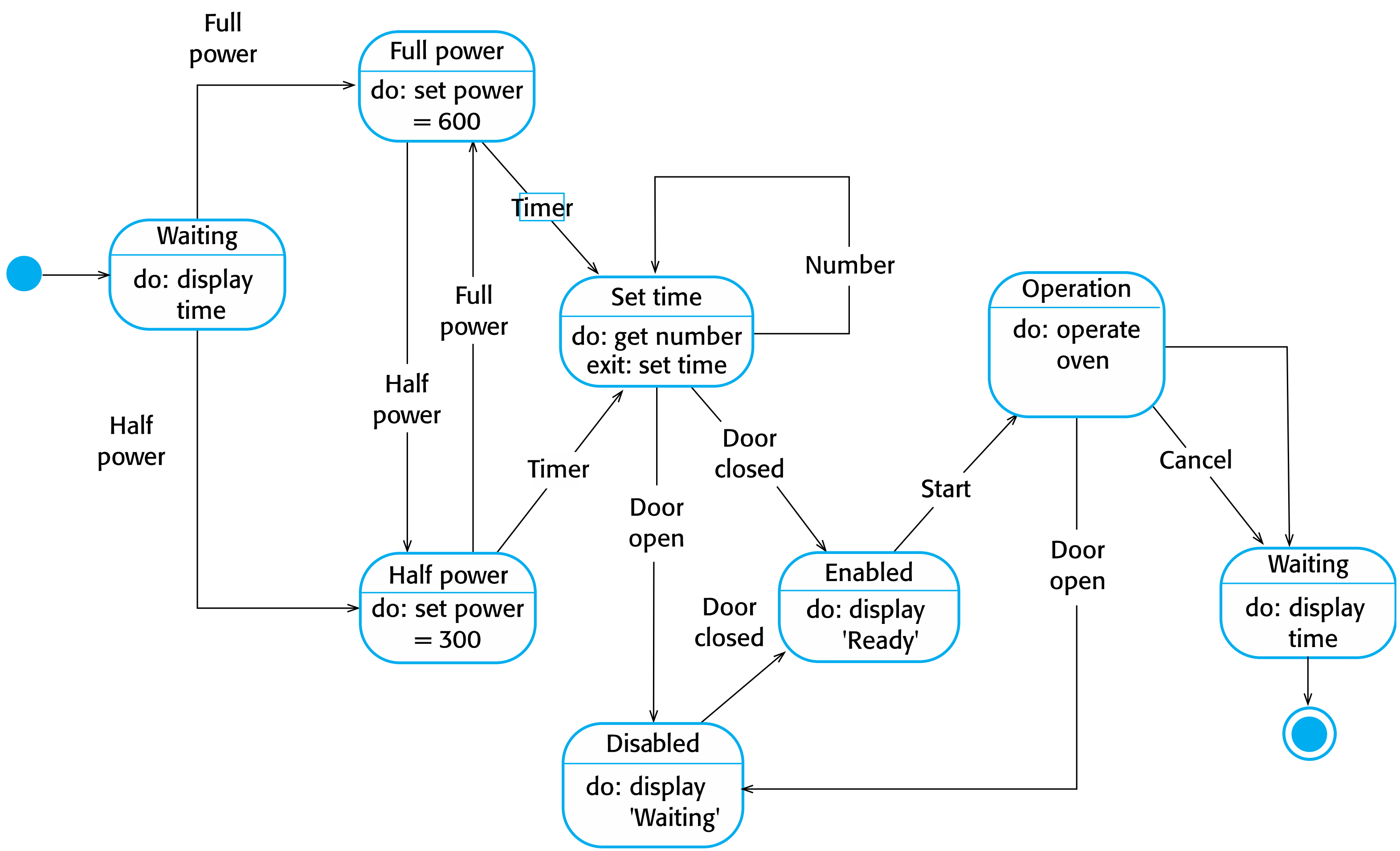
State machine models



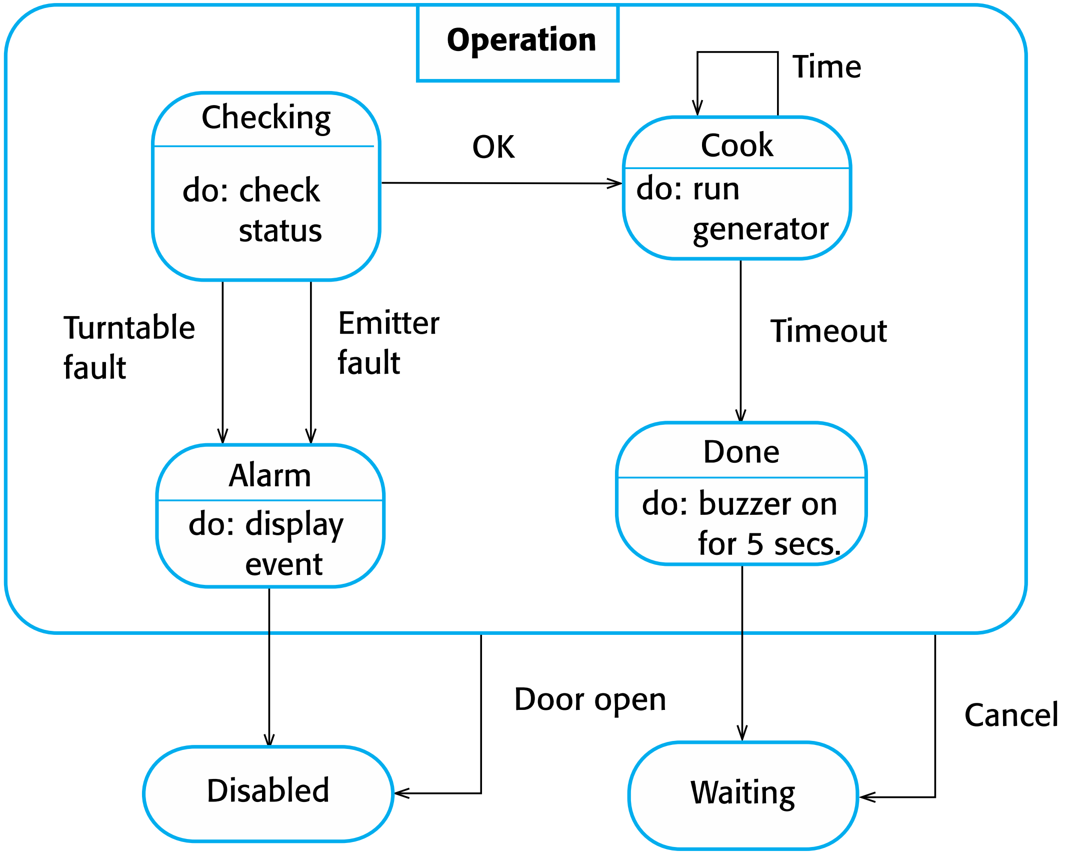
- **State machine models**:
  - model the behaviour of the system in response to external and internal events.
  - When an event occurs, the system moves from one state to another
- They show the system's responses to stimuli
  - **often used for modeling real-time systems.**

State diagram of a microwave oven





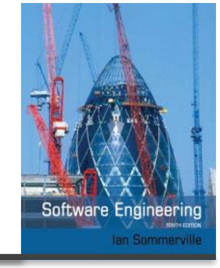
Microwave oven operation



States and stimuli for the microwave oven (a)

State	Description
Waiting	The oven is waiting for input. The display shows the current time.
Half power	The oven power is set to 300 watts. The display shows 'Half power'.
Full power	The oven power is set to 600 watts. The display shows 'Full power'.
Set time	The cooking time is set to the user's input value. The display shows the cooking time selected and is updated as the time is set.
Disabled	Oven operation is disabled for safety. Interior oven light is on. Display shows 'Not ready'.
Enabled	Oven operation is enabled. Interior oven light is off. Display shows 'Ready to cook'.
Operation	Oven in operation. Interior oven light is on. Display shows the timer countdown. On completion of cooking, the buzzer is sounded for five seconds. Oven light is on. Display shows 'Cooking complete' while buzzer is sounding.

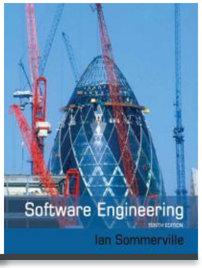
States and stimuli for the microwave oven (b)



Stimulus	Description
Half power	The user has pressed the half-power button.
Full power	The user has pressed the full-power button.
Timer	The user has pressed one of the timer buttons.
Number	The user has pressed a numeric key.
Door open	The oven door switch is not closed.
Door closed	The oven door switch is closed.
Start	The user has pressed the Start button.
Cancel	The user has pressed the Cancel button.

Model-driven engineering

Model-driven engineering



- **Model-driven engineering (MDE)** is an approach to software development where **models rather than programs are the principal outputs of the development process.**
- The programs that execute on a hardware/software platform are then **generated automatically from the models.**

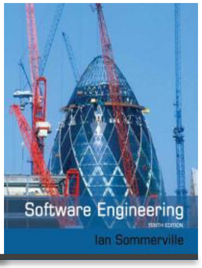
Types of model

- ✧ **A computation independent model (CIM)**
  - These model the important domain abstractions used in a system.
- ✧ **A platform independent model (PIM)**
  - These model the operation of the system without reference to its implementation.
  - static system structure
- ✧ **Platform specific models (PSM)**
  - These are transformations of the platform-independent model with a separate PSM for each application platform.

Usage of model-driven engineering

- **Model-driven engineering** is still at an early stage of development
- **Pros**
  - **Higher productivity**: Much code is automatically generated.
  - **Reduced errors**: Because systems are generated from models, human coding errors decrease.
  - Platform independence: A model can generate Java, C++, Python, C #
- **Cons**
  - **Tool complexity:** MDE tools can be difficult to use.
  - **Over-engineering risk**: Some projects become too abstract and complex.

Model driven architecture



- ✧ **Model-driven architecture (MDA)** - more general model-driven engineering

- ✧ **Models at different levels of abstraction are created**.
  - It is possible to generate a working program without manual intervention.

Model Transformations

- Models can be transformed into other models.
  - Example
- Platform Independent Model (PIM)

  ↓

  Transformation

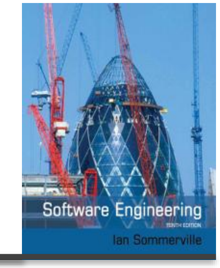
  ↓

  Platform Specific Model (PSM)

  ↓

  Generated Code

Relationship Between MDE and AI



✧ A new trend is **AI-assisted model generation**.

- These model the important domain abstractions used in a system.

✧ AI tools can now:

- Text requirement → generate UML models
  UML models → generate code.
- This is an emerging research direction.

Future of MDE

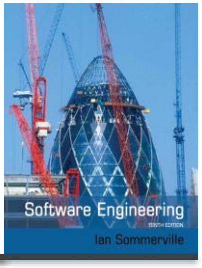
✧ The future direction is **AI-Driven Model Engineering**:

Natural Language Requirement
↓
AI generates system models
↓
Automatic code generation
↓
Self-adapting software

✧ This area is called:

- **AI-Driven Software Engineering (AI-SE)**.

How Far MDE Has Progressed (Current Status)



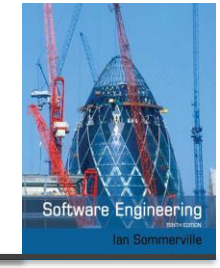
Stage	Status
Academic research	Very mature
Tool ecosystem	Mature
Industrial adoption	Moderate
Startups / web apps	Low
Safety-critical systems	Very high

- MDE is **well developed academically but selectively adopted industrially**.

Key points

- ✧ A model is an abstract view of a system that ignores system details. Complementary system models can be developed to show the system's context, interactions, structure and behavior.
- ✧ Context models show how a system that is being modeled is positioned in an environment with other systems and processes.
- ✧ Use case diagrams and sequence diagrams are used to describe the interactions between users and systems in the system being designed. Use cases describe interactions between a system and external actors; sequence diagrams add more information to these by showing interactions between system objects.
- ✧ Structural models show the organization and architecture of a system. Class diagrams are used to define the static structure of classes in a system and their associations.

Key points



- Behavioral models are used to describe the dynamic behavior of an executing system. This behavior can be modeled from the perspective of the data processed by the system, or by the events that stimulate responses from a system.
- Activity diagrams may be used to model the processing of data, where each activity represents one process step.
- State diagrams are used to model a system's behavior in response to internal or external events.
- Model-driven engineering is an approach to software development in which a system is represented as a set of models that can be automatically transformed to executable code.